Backpropagation as a Linear Solve

Mikkel Paltorp (mpasc@dtu.dk), 5 October 2026
Department of Applied Mathematics and Computer Science, Technical University of Denmark

In the previous post on backpropagation[1], we showed that backpropagation is a special case of the adjoint method. In this post we skip the adjoint description and instead derive the same result directly from the chain rule.

Contents

1. The forward relation
2. Differentiating the network
3. A scalar loss

[1] Earlier post: *Backpropagation is the Adjoint Method*. mipals.github.io/blog/2025/10-25-adjoint-method/

1. The forward relation

We denote the data by u_0 and the parameters by $\theta \in \mathbb{R}^p$. The network has N layers, each of which is a function Φ_i that takes as input outputs from previous layers $(u_{i-1}, \dots, u_0)$ as well as the parameters (θ), and produces the next layer output u_i. The full forward pass, including all layer outputs, can be expressed as

$$\underbrace{\begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix}}_{u(\theta)} = \underbrace{\begin{bmatrix} \Phi_1(u_0, \theta) \\ \Phi_2(u_1, u_0, \theta) \\ \vdots \\ \Phi_N(u_{N-1}, \dots, u_0, \theta) \end{bmatrix}}_{\Phi(u(\theta), \theta)} \in \mathbb{R}^d, \tag{1}$$

where $d = \sum_{i=1}^{N} n_i$ and n_i is the size of u_i.

We introduce the projection $P \in \mathbb{R}^{r \times d}$ that takes out relevant layer-outputs, i.e.[2]

$$u_P(\theta) = Pu(\theta). \tag{2}$$

[2] Often P just picks out the last layer, i.e. $u_P = u_N = Pu(\theta), \quad P = \begin{pmatrix} 0 & \dots & 0 & I_r \end{pmatrix}$.

Using this the loss function can be written as

$$\mathcal{J}(\theta) = \ell(u_P(\theta)) + \mu \mathcal{R}(\theta), \tag{3}$$

where ℓ is the data loss and $\mathcal{R}$ is a regularization term.

2. Differentiating the network

Applying the chain rule to Equation 1, we have

$$\frac{\mathrm{d}u}{\mathrm{d}\theta} = \frac{\partial \Phi}{\partial u}\frac{\mathrm{d}u}{\mathrm{d}\theta} + \frac{\partial \Phi}{\partial \theta}, \quad \text{hence} \quad \frac{\mathrm{d}u}{\mathrm{d}\theta} = \left(\underbrace{I - \frac{\partial \Phi}{\partial u}}_{L} \right)^{-1} \underbrace{\frac{\partial \Phi}{\partial \theta}}_{M^\top}. \tag{4}$$

Applying the projection P, it further follows that

$$\frac{\mathrm{d}u_P}{\mathrm{d}\theta} = PL^{-1}M^\top. \tag{5}$$

At this point it is worth looking at the structure of the matrices. As the ith layer depends only on previous outputs, the matrix L is block lower-triangular with identity blocks on the diagonal[3]. The triangular structure makes it efficient to solve the linear system

$$L^\top \Lambda = P^\top, \quad \Lambda \in \mathbb{R}^{d \times r}. \tag{6}$$

[3] The sparsity pattern of the strictly lower/upper part of a (block) triangular matrix is a directed acyclic graph (DAG), which is often how backpropagation is introduced.

Using Λ it follows that

$$\frac{\mathrm{d}u_P}{\mathrm{d}\theta} = \Lambda^\top M^\top. \tag{7}$$

In practice, as seen in the next section, one rarely needs to form $\frac{\mathrm{d}u_P}{\mathrm{d}\theta}$ as a full matrix; it suffices to define products with it.

3. A scalar loss

Applying the chain rule to Equation 3, we have

$$\nabla_\theta \mathcal{J} = \left(\frac{\mathrm{d}u_P}{\mathrm{d}\theta}\right)^\top \nabla_{u_P}\ell + \mu\nabla_\theta \mathcal{R} = ML^{-\top}P^\top \nabla_{u_P}\ell + \mu\nabla_\theta \mathcal{R}. \tag{8}$$

Given that $\nabla_{u_P}\ell$ is a vector of length r we have that the product $P^\top\nabla_{u_P}\ell$ is a vector of length d. It is therefore of interest to first form $P^\top\nabla_{u_P}\ell$ and then solve $L^\top\lambda = P^\top\nabla_{u_P}\ell$, so that

$$\nabla_\theta \mathcal{J} = M\lambda + \mu\nabla_\theta \mathcal{R}, \tag{9}$$

which is equivalent to the final result of the adjoint method (see the previous post on backpropagation[4]). Note that λ here is the negative of the adjoint variable of the previous post, which absorbs the sign of $\frac{\partial f}{\partial \theta} = -M^\top$ there.

[4] Earlier post: *Backpropagation is the Adjoint Method*. mipals.github.io/blog/2025/10-25-adjoint-method/